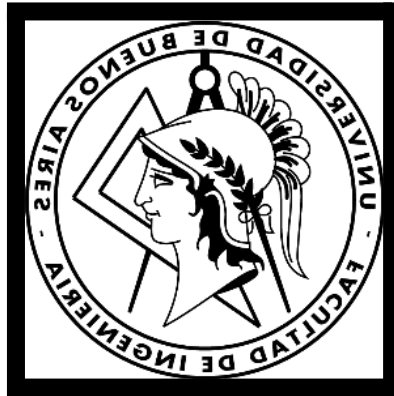


TP I

TEORÍA DE ALGORITMOS - ECHEVARRÍA



LUCA FRISONI - 113497

SOFÍA MARTIGNETTI - 101930

VALENTINA DÍAZ RACIOPPI - 110998

LUCAS VILLARRUBIA - 108230

MATIAS LEGUIZAMON - 108109

Índice

Problema 1 - División y conquista.....	2
Análisis Complejidad.....	2
Gráficos.....	4
Otras preguntas teóricas.....	6
Problema 1 - División y conquista (Alternativa).....	7
Problema 2 - Greedy.....	10
Supuestos y Consideraciones de Diseño.....	10
¿Qué problema resuelve el código encontrado?.....	10
¿Qué algoritmo conocido replica dicho código?.....	10
Corrección en la implementación en Python.....	10
¿Cuál es el uso de las principales estructuras de datos del programa?.....	11
¿Por qué se puede considerar que es un algoritmo Greedy? ¿Cuál es la regla que se aplica?.....	12
Complejidad temporal y espacial.....	12
Complejidad temporal.....	12
Complejidad espacial.....	12
Problema 3 - Backtracking.....	13
Supuestos y Consideraciones de Diseño.....	13
Laberinto.....	13
Aspiradora.....	13
Condiciones de Búsqueda.....	13
Análisis del Problema.....	13
Análisis Preliminar.....	13
El Problema con las Limitaciones Impuestas.....	14
Soluciones.....	14
Propuestas de Implementación.....	14
Estructuras y Memoria.....	15
Seguimiento: Cómo funciona la poda de ramas.....	15
Pseudocódigo.....	16
Análisis de Complejidad.....	17
Complejidad Temporal.....	17
Complejidad Espacial.....	17
Pero, por DLS.....	18
Resultados y Conclusiones.....	18
Problema 4 - Programación Dinámica.....	21
Supuestos, condiciones y limitaciones.....	21
Diseño.....	21
Seguimiento.....	22
Análisis de complejidad.....	23
Sets de datos.....	23
Tiempos de ejecución.....	23
Referencias bibliográficas.....	24

Problema 1 - División y conquista

ANÁLISIS COMPLEJIDAD

Alcance del Análisis

El análisis de complejidad temporal se realiza sobre la función *moneda_falsa()*, ya que constituye el algoritmo de **División y Conquista** encargado de resolver el problema principal.

Las funciones auxiliares, como *generar_monedas()*, no forman parte del núcleo algorítmico a analizar, dado que únicamente se utilizan para la generación de datos de prueba

Descripción del Algoritmo

El algoritmo *moneda_falsa()* recibe:

- Un arreglo de monedas
- Un contador de pesadas
- Un desplazamiento (offset) para mantener el índice original

En cada llamada:

1. Se divide el conjunto de monedas en dos subconjuntos de igual tamaño.
2. Se calcula el peso total de cada subconjunto.
3. Se identifica cuál de los dos subconjuntos contiene la moneda falsa (el más liviano).
4. Se aplica recursivamente el mismo procedimiento sobre dicho subconjunto.

En caso de que ambos subconjuntos tengan el mismo peso, la moneda falsa se encuentra en el subconjunto restante (cuando la cantidad de monedas es impar).

El caso base ocurre cuando el subconjunto tiene una única moneda.

Explicación complejidad

En cada paso del algoritmo se realizan las siguientes operaciones:

- División del problema en un subconjunto de tamaño $n/2$
- Cálculo del peso de subconjuntos mediante la función *sum()*, lo cual implica recorrer los elementos

La recurrencia tiene la forma:

$$T(n) = a \cdot T(n/b) + f(n) \text{ } ^1$$

Donde:

- $a = 1$ (se realiza una sola llamada recursiva)
- $b = 2$ (el problema se reduce a la mitad)
- $f(n) = O(n)$ (costo de calcular las sumas)

Por lo tanto, la recurrencia que modela el algoritmo es:

$$T(n) = T(n/2) + O(n)$$

Calculamos $\log_b(a) = c$

$$\log_b(a) = \log_2(1) = 0$$

Comparando

- Como $n^c < f(n) \rightarrow n^0 < n$

La complejidad por **Teorema maestro** es:

$$T(n) = O(n) \text{ }^1$$

Análisis teórico alternativo

Si se considera un modelo teórico en el cual se dispone de una **balanza ideal** capaz de comparar el peso de dos subconjuntos en tiempo constante $O(1)$, la recurrencia sería:

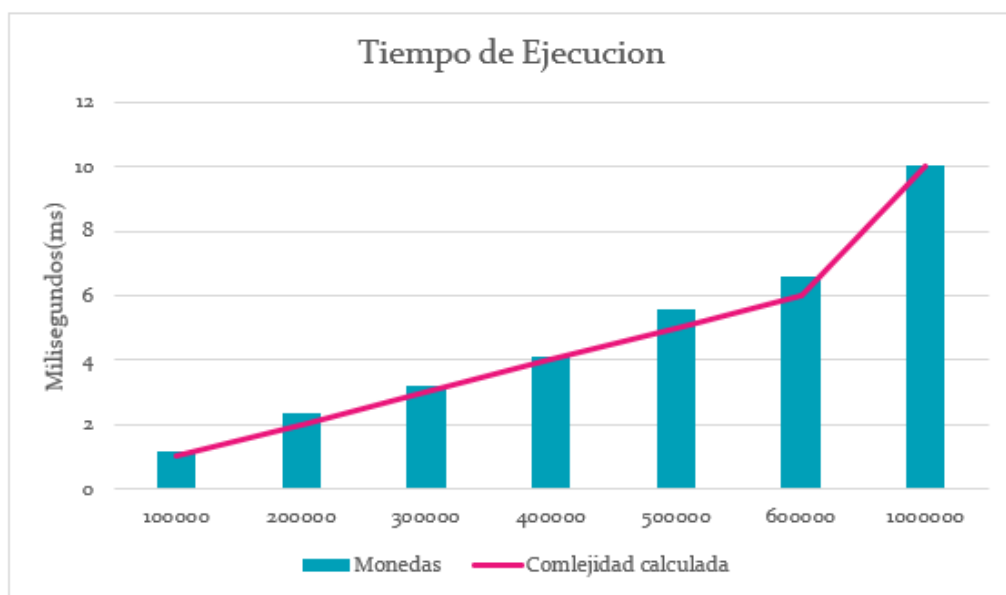
$$T(n) = T(n/2) + O(1)$$

Lo cual se resuelve como:

$$T(n) = O(\log n) \text{ }^2$$

GRÁFICOS

Tiempo de Ejecución



Se seleccionaron como casos de prueba distintos tamaños de entrada n , tomando valores desde 100.000 hasta 1.000.000 de monedas, incrementando de forma uniforme (100.000, 200.000, 300.000, 400.000, 500.000, 600.000 y 1.000.000).

Dado que la complejidad temporal del algoritmo fue determinada teóricamente como $O(n)$, se construyó una referencia gráfica basada en una función lineal, independiente de los tiempos medidos. Para ello, se asignó un valor proporcional a n , considerando que cada incremento de 100.000 monedas corresponde a un incremento unitario en la función de referencia. Por ejemplo:

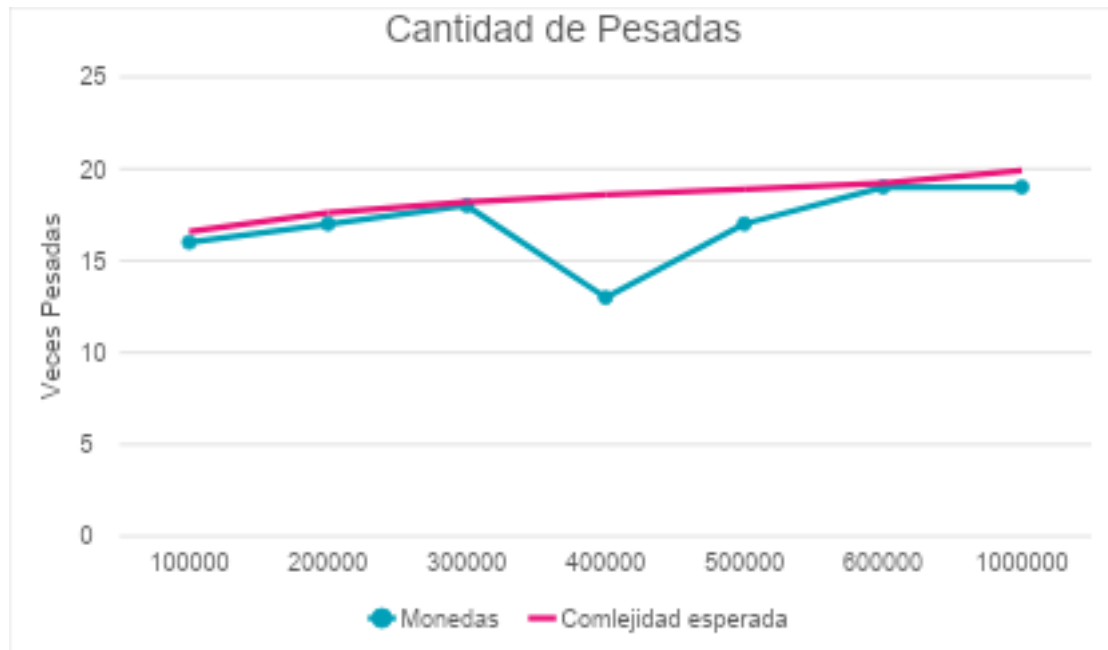
- 100.000 $\rightarrow$ 1
- 500.000 $\rightarrow$ 5
- 1.000.000 $\rightarrow$ 10

Esto permite representar visualmente una recta de crecimiento lineal ideal.

Al observar el gráfico, se aprecia que los tiempos de ejecución medidos siguen una tendencia claramente lineal en función del tamaño de entrada. En particular, el crecimiento del tiempo acompaña proporcionalmente el aumento en la cantidad de monedas, alineándose de manera consistente con la recta de referencia $O(n)$.

Incluso en el salto entre 600.000 y 1.000.000 de monedas, se mantiene este comportamiento lineal, lo que refuerza la validez del análisis teórico realizado. Si bien pueden existir pequeñas variaciones debido a factores propios de la ejecución (como el entorno o la implementación), la tendencia general confirma que el algoritmo presenta un crecimiento lineal.

Cantidad de Pesadas



Para este análisis se utilizaron los mismos tamaños de entrada que en el gráfico anterior, con valores de n comprendidos entre 100.000 y 1.000.000 de monedas.

Al observar los resultados obtenidos para la cantidad de pesadas, se aprecia que los valores no siguen una tendencia lineal. En cambio, presentan pequeñas variaciones y aparentan ser irregulares. Sin embargo, este comportamiento no es aleatorio, sino que responde directamente a la naturaleza del algoritmo.

En cada iteración, el problema se reduce aproximadamente a la mitad, lo que implica que la cantidad de pasos necesarios para encontrar la moneda falsa crece de forma logarítmica en función del tamaño de entrada. En consecuencia, la complejidad en términos de cantidad de pesadas es del orden:

$$O(\log_2(n))$$

Este tipo de crecimiento se caracteriza por ser muy lento en comparación con el crecimiento lineal. Por este motivo, incluso incrementos grandes en el tamaño de entrada producen aumentos pequeños en la cantidad de pesadas.

Se observa un valor significativamente menor en uno de los casos analizados (por ejemplo, 13 pesadas para 400.000 monedas), en comparación con los valores esperados cercanos a 17 o 18.

Si bien en ciertos casos particulares la moneda falsa puede encontrarse en subconjuntos más pequeños (por ejemplo, cuando interviene el conjunto restante en divisiones no exactas), lo que puede reducir levemente la cantidad de iteraciones necesarias, estas situaciones no son las más frecuentes.

En este sentido, el valor observado puede considerarse un caso poco representativo dentro del conjunto de mediciones realizadas. De hecho, al repetir el experimento múltiples veces para el mismo tamaño de entrada, los resultados obtenidos se mantienen consistentemente en torno a los valores esperados.

En conclusión, aunque el gráfico no presenta una forma perfectamente uniforme, los resultados son consistentes con un comportamiento logarítmico, validando el análisis teórico del algoritmo.

OTRAS PREGUNTAS TEÓRICAS

1. **¿Es correcto vincular el tiempo de ejecución con la cantidad de pesadas?**

¿Por qué?

No es completamente correcto vincular directamente el tiempo de ejecución con la cantidad de pesadas.

Si bien ambas métricas están relacionadas con el comportamiento del algoritmo, representan aspectos distintos de su ejecución.

Por un lado, la cantidad de pesadas refleja la cantidad de iteraciones del algoritmo de División y Conquista, y crece de forma logarítmica $O(\log_2(n))$, ya que en cada paso el problema se reduce aproximadamente a la mitad.

Por otro lado, el tiempo de ejecución depende no solo de la cantidad de pesadas, sino también del costo de las operaciones realizadas en cada iteración. En este caso, el cálculo del peso de los subconjuntos mediante la función `sum()` implica recorrer los elementos, lo que introduce un costo lineal en cada nivel.

Como consecuencia, aunque la cantidad de pesadas es logarítmica, el tiempo total de ejecución resulta ser lineal $O(n)$.

En conclusión, no es correcto vincular directamente ambas métricas, ya que la cantidad de pesadas mide la cantidad de decisiones del algoritmo, mientras que el tiempo de ejecución refleja el costo total de todas las operaciones realizadas.

PROBLEMA 1 - DIVISIÓN Y CONQUISTA (ALTERNATIVA)

Descripción del Algoritmo

La idea es dividir el arreglo de monedas en 3 grupos lo más parejos posible. Los dos primeros tienen el mismo tamaño ($n/3$ cada uno) y lo que sobra queda en el tercero. Se comparan los dos primeros grupos:

- Si el grupo 1 pesa menos, la falsa está ahí.
- Si el grupo 2 pesa menos, la falsa está en el grupo 2.
- Si pesan igual, la falsa está en el grupo 3.

Se aplica recursivamente sobre el grupo donde quedó la falsa hasta que queda una sola moneda. Elegí dividir en 3 en vez de 2 porque una pesada tiene 3 resultados posibles, así se aprovecha mejor cada comparación y se hacen menos pesadas en total.

Análisis de complejidad

Cantidad de pesadas

En cada llamada recursiva se hace una pesada y el problema se reduce a un tercio. La recurrencia es:

$$T(n) = T(n/3) + O(1)$$

Entonces la cantidad de pesadas es $O(\log_3 n)$ ¹.

Tiempo de ejecución

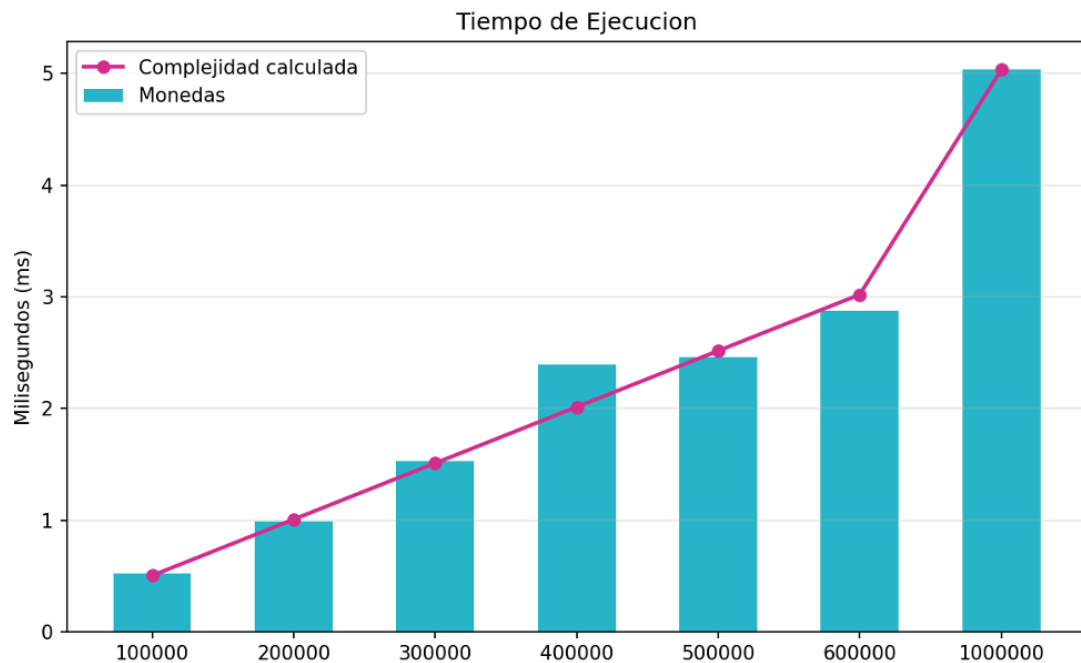
En la implementación real, calcular el peso de un grupo con `sum()` recorre los elementos, así que cada pesada cuesta $O(n)$. La recurrencia real queda:

$$T(n) = T(n/3) + O(n)$$

Aplicando el Teorema Maestro ($a = 1$, $b = 3$, $f(n) = O(n)$), se cumple el tercer caso y la complejidad temporal es $O(n)$ ¹.

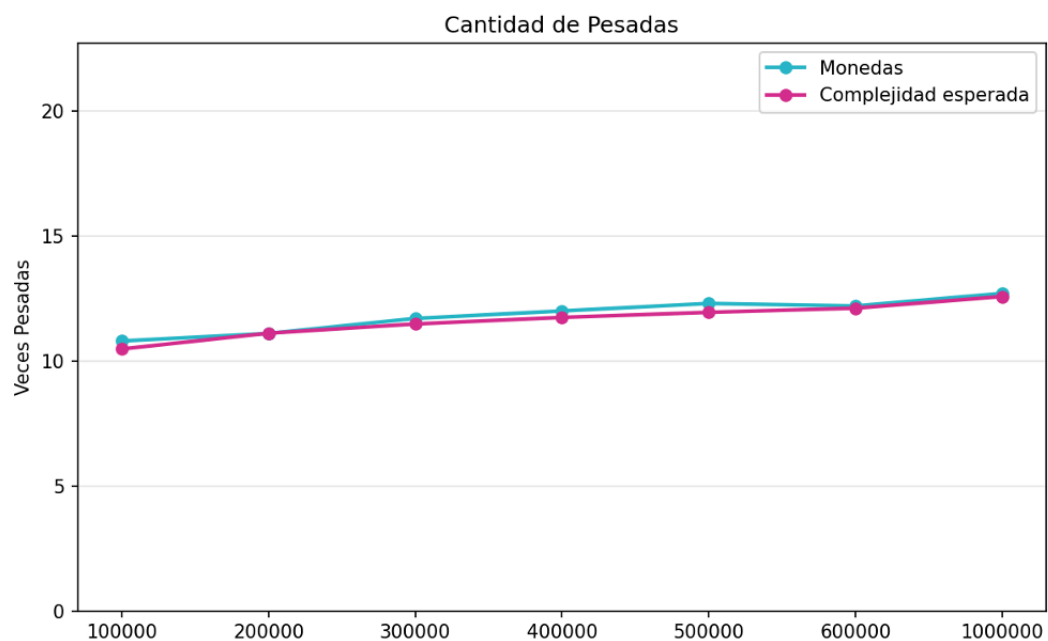
Gráficos

Tiempo de ejecución



Se probó con tamaños entre 100.000 y 1.000.000 de monedas. El tiempo crece de forma lineal con n , tal como predice el análisis teórico $O(n)$. La línea rosa muestra la curva lineal de referencia y las barras los tiempos medidos, que acompañan bastante bien la tendencia.

Cantidad de pesadas



La cantidad de pesadas crece muy lentamente con n , siguiendo a $\log_3(n)$. Por eso incluso multiplicando por 10 el tamaño de entrada de 100.000 a 1.000.000, las pesadas apenas pasan de 11 a 13. Las pequeñas variaciones se deben a que, según la posición de la falsa, a veces se entra antes al caso base de 2 monedas y se ahorra o se gasta alguna pesada.

¿Es correcto vincular tiempo con cantidad de pesadas?

No directamente. Son dos cosas distintas:

- La cantidad de pesadas mide cuántas comparaciones hace el algoritmo y crece de forma logarítmica porque en cada paso se descarta $\frac{2}{3}$ del problema.
- El tiempo de ejecución depende también de cuánto cuesta hacer cada pesada. Como acá el peso de un grupo se calcula con `sum()` (recorriendo los elementos), cada pesada tiene un costo lineal.

Resultado: las pesadas son $O(\log n)$ pero el tiempo total es $O(n)$. Solo serían equivalentes si se asumiera una balanza ideal que compara pesos en $O(1)$

Conclusiones

- Dividir en 3 grupos reduce la cantidad de pesadas a $O(\log_3 n)$, aprovechando que la balanza tiene 3 resultados posibles.
- El tiempo de ejecución es $O(n)$ porque cada pesada recorre los elementos con `sum()`
- Los resultados medidos son consistentes con el análisis teórico.
- Pesadas y tiempo no son equivalentes: miden aspectos distintos del algoritmo.

Problema 2 - Greedy

SUPUESTOS Y CONSIDERACIONES DE DISEÑO

¿Qué problema resuelve el código encontrado?

El programa permite al usuario ingresar un grafo mediante una matriz de costos y posteriormente seleccionar un vértice de origen. A partir de este vértice, se ejecuta el algoritmo de Dijkstra para calcular las distancias mínimas hacia todos los demás nodos.

El sistema permite repetir la consulta para distintos vértices de origen sin necesidad de reingresar el grafo, lo que facilita el análisis de caminos mínimos desde múltiples puntos.

Por lo tanto, el código resuelve el problema de **cálculo de caminos mínimos desde un vértice origen hacia todos los demás vértices de un grafo ponderado**.

Dado un grafo representado mediante una matriz de costos y un vértice inicial V, el programa calcula la distancia mínima desde dicho vértice hacia cada uno de los restantes nodos del grafo.

Este problema es conocido como **Single Source Shortest Path (SSSP)**¹.

El resultado final consiste en un conjunto de distancias mínimas desde el origen a cada vértice alcanzable.

¿Qué algoritmo conocido replica dicho código?

Tal cómo se mencionó en el apartado anterior, el código implementa el **algoritmo de Dijkstra**³.

Este algoritmo es un método clásico para resolver el problema de caminos mínimos en grafos con pesos no negativos. Su funcionamiento se basa en una estrategia iterativa que selecciona, en cada paso, el vértice más cercano al origen que aún no ha sido procesado, y actualiza las distancias hacia sus vecinos.

La estructura general del código coincide con las etapas características de Dijkstra:

1. Inicialización de distancias desde el nodo origen
2. Selección iterativa del nodo más cercano no procesado
3. Relajación de aristas
4. Repetición hasta procesar todos los nodos

Corrección en la implementación en Python

En el código, la variable **U** se inicializa con un valor alto (15000), lo cual sugiere que representa una cota superior para las distancias.

Sin embargo, dentro del mismo bloque se le asigna el valor del índice del nodo seleccionado, por lo que pasa a representar un vértice.

Esta reutilización de la variable introduce una ambigüedad conceptual, ya que se compara una distancia (**DIST(J)**) con un índice (**U**), lo cual no es consistente con la lógica del algoritmo de Dijkstra.

Por tal motivo, en nuestra implementación del código en Python se optó por crear una nueva variable para el valor mínimo (**min_val**) y otra para índice (**U**).

Se implementó una versión en Python que replica el comportamiento del programa original en BASIC, incluyendo el ingreso interactivo de datos y la posibilidad de realizar múltiples consultas sobre el mismo grafo. Al código, se añadió:

- Corrida de testeo con datos de prueba (tabla provista en la consigna) que saltea el ingreso manual.
- Validación mínima sobre tipo de datos en caso de ingreso erróneo.
- Modularización del código en funciones independientes para mejorar la legibilidad y facilitar su análisis.

La salida del programa presenta los vértices en orden creciente, lo cual responde a una decisión de formato y no al funcionamiento del algoritmo.

El algoritmo de Dijkstra no ordena los nodos por distancia en su salida, sino que calcula las distancias mínimas, quedando el orden determinado por la estructura de impresión.

¿Cuál es el uso de las principales estructuras de datos del programa?

El programa utiliza tres estructuras principales:

Matriz de costos **COST(N, N)**

- Representa el grafo ponderado.
- Cada entrada indica el costo de ir de un nodo a otro.
- Se utiliza un valor grande (15000) para representar ausencia de conexión.
- Se fuerza simetría en la matriz, lo que implica que el grafo es no dirigido.

Vector **DIST(N)**

- Almacena la distancia mínima conocida desde el nodo origen hacia cada vértice.
- Inicialmente se carga con los costos directos desde el origen.
- Se actualiza durante la ejecución mediante el proceso de relajación.

Vector **SOL(N)**

- Indica si un nodo ya fue procesado (visitado).
- Permite evitar reprocesar nodos y garantiza la lógica del algoritmo greedy.

¿Por qué se puede considerar que es un algoritmo Greedy? ¿Cuál es la regla que se aplica?

El algoritmo es considerado **greedy (voraz)** porque en cada iteración toma una decisión local óptima sin reconsiderar decisiones anteriores.

Para este caso, la regla Greedy del algoritmo es: **En cada paso, se selecciona el vértice no procesado con la menor distancia acumulada desde el nodo origen.**

Esta estrategia es válida porque **en grafos con pesos no negativos:**

- Una vez que se selecciona el nodo con menor distancia, dicha distancia es definitiva.
- No existe un camino posterior que pueda mejorarla.

Esto garantiza que las decisiones locales conduzcan a una solución global óptima.

COMPLEJIDAD TEMPORAL Y ESPACIAL

Complejidad temporal

El algoritmo realiza:

- N iteraciones principales
- En cada iteración:
 - Búsqueda lineal del mínimo $\rightarrow O(N)$
 - Actualización de distancias $\rightarrow O(N)$

Por lo tanto:

$$O(N^2)$$

Complejidad espacial

- Matriz de costos $\rightarrow O(N^2)$
- Vectores auxiliares $\rightarrow O(N)$

Por lo tanto:

$$O(N^2)$$

Problema 3 - Backtracking

SUPUESTOS Y CONSIDERACIONES DE DISEÑO

Para abordar el problema de la aspiradora, primero se enuncian las reglas y limitaciones propuestas y derivadas de la consigna para la implementación de un algoritmo correcto y acotado.

Laberinto

- Rectangular, una matriz de $(m \times n)$
- Hay sólo una posición de comienzo y sólo una salida
- La salida siempre va a estar ubicada en una pared externa
- La entrada puede estar en cualquier posición y se considera un espacio vacío (para que sea legal ubicar la aspiradora ahí)
- La matriz tiene su perímetro cubierto por paredes, excepto por la salida

Aspiradora

- Se ubica en una sola posición de la matriz lógicamente
- Sólo puede ubicarse en espacios vacíos
- Tendrá una dirección para la cuál se esté moviendo, inicialmente tiene que estar determinada de alguna manera
- Sólo puede “mirar” hacia adelante una sola casilla, no “ve lo que hay más allá” hasta que avanza

Condiciones de Búsqueda

- Puede haber laberintos sin solución posible, el algoritmo tiene que poder identificar esto y manejar el error para no caer en loops infinitos

ANÁLISIS DEL PROBLEMA

Análisis Preliminar

Además de modelar el laberinto como una matriz para facilitar la visualización se puede abstraer como un grafo no dirigido: cada casilla vacía explorable representaría un vértice y los caminos hacia casillas contiguas representan las aristas.

Con este modelo, y sabiendo que la aspiradora ocupa un lugar físico en el espacio, la búsqueda a lo ancho de un árbol (BFS) resulta inadecuada. La aspiradora debería moverse innecesariamente para seguir el recorrido concéntrico de ese tipo de algoritmo.

Por lo tanto, es óptimo utilizar un algoritmo de búsqueda en profundidad (DFS) para avanzar por ramas hasta encontrar caminos sin salida antes de retroceder.

Existe una analogía con un algoritmo clásico: el Algoritmo de Trémaux, un método diseñado empíricamente para resolver laberintos físicos. La regla fundamental de Trémaux es dejar marcas en los caminos a medida que se transitan para no recorrer un mismo pasillo dos veces y evitar ciclos infinitos. En este caso, las marcas para retroceder pueden consistir en guardar en memoria las casillas visitadas por la aspiradora.

El Problema con las Limitaciones Impuestas

El estado de la aspiradora no se define sólo por su posición (x, y), sino por el par (Posición, Orientación), siempre mira hacia una dirección. Esto implica que para explorar las cuatro celdas adyacentes a su posición actual, el algoritmo debe incurrir en costos de rotación física. Dado que el dispositivo solo puede girar 90° hacia la izquierda, un cambio de dirección puede costar entre 1 y 3 operaciones de giro.

Al momento de “podar” una rama o camino ya visitado, el retroceso debe ser físico. Para esto y para cumplir con el avance de una DFS, el diseño del algoritmo debe tener en cuenta:

- Las direcciones de casillas que ya se visitaron.
- La reversa y realineación con la dirección inicial con la que “entró” a una rama o camino.
- La complejidad temporal que conlleva estas operaciones físicas, para la que tiene que ver completamente la topología del laberinto.

SOLUCIONES

Propuestas de Implementación

Con el objetivo de plasmar la diferencia entre complejidades y comparar soluciones para el problema, se implementó el mismo algoritmo de manera recursiva e iterativa.

Para parametrizar ambos métodos de implementación, se planteó la Búsqueda Limitada por Profundidad (DLS, Depth-Limited Search). Se traduce en una restricción del mundo físico (batería, tiempo o desgaste mecánico de la aspiradora) en una optimización computacional. En un DFS puro, la aspiradora podría explorar ramas masivas antes de retroceder. Al incorporar un límite o umbral máximo de profundidad (L), el algoritmo fuerza un early pruning de estas ramas. Como suposición, debería ahorrar esfuerzo en operaciones mecánicas innecesarias en laberintos densos, e impacta en la complejidad espacial.

Enfoque Recursivo

La implementación recursiva es la traducción más directa de DFS. Delega la gestión del estado (la posición, la dirección de entrada y la profundidad actual) al Call Stack de Python.

Como ventaja, el código es mucho más conciso y fácil de leer. Por el contrario, la desventaja es el límite de la memoria: En el peor caso (un laberinto donde el camino es un único pasillo largo y serpenteante), la profundidad de la recursión deberá crecer significativamente. Python tiene un límite de seguridad en su Call Stack. Si bien puede sobreescribirse, depender de la memoria del sistema operativo para almacenar el estado de un recorrido extenso es una vulnerabilidad y puede resultar en Stack Overflow.

Enfoque Iterativo

Para mitigar la limitación espacial del enfoque recursivo, se propone una segunda solución que transforma el algoritmo en iterativo. Se instancia una estructura de

memoria propia (un Stack explícito) que reside en el Heap, el cual es muchísimo más grande y flexible que el Call Stack.

Con este enfoque, no hay riesgo de colapso de memoria, limitándose únicamente por la memoria RAM disponible. Pero, exige gestionar manualmente los "push" y "pop" de la pila, además de depender de while loops para efectuar el backtracking.

Estructuras y Memoria

- **Conjunto *visited* (Hash Set):** Se utiliza una tabla hash (la estructura set() en Python) para almacenar las tuplas de coordenadas (Fila, Columna) ya exploradas. Esto garantiza que la validación se resuelva con un costo temporal de $O(1)$ en promedio, evitando las búsquedas lineales de $O(V)$ que ocurrirían al usar arreglos comunes.
- **Pila de Exploración (LIFO Stack):** Exclusiva del enfoque iterativo. Apilar y desapilar el estado actual (fila, columna, direccion_entrada, profundidad) tiene un costo de $O(1)$.
- **DTO Nodo (PathNode):** Exclusivo del método iterativo. Estructura de datos con información de movimiento de la aspiradora durante el DFS.
- **Map Estático de Direcciones:** Para evitar múltiples estructuras condicionales al calcular las coordenadas futuras según la orientación de la aspiradora, se emplea un diccionario en memoria DIR_OPS que mapea directamente un número entero (0 a 3, representando la dirección) con un par de incrementos lógicos. Esto asegura cálculos de avance en $O(1)$.

Seguimiento: Cómo funciona la poda de ramas

1. La aspiradora comienza en (1, 1). Apila su estado inicial y marca la celda como visitada.

Pila: [(F:1, C:1, DirEntrada:3, Prof:0)] | Visitados: {(1,1)}

2. Al mirar hacia adelante (Derecha), ve una celda vacía 'O' en (1, 2). Avanza.

Pila: [(1,1,3,0), (1,2,3,1)] | Visitados: {(1,1), (1,2)}

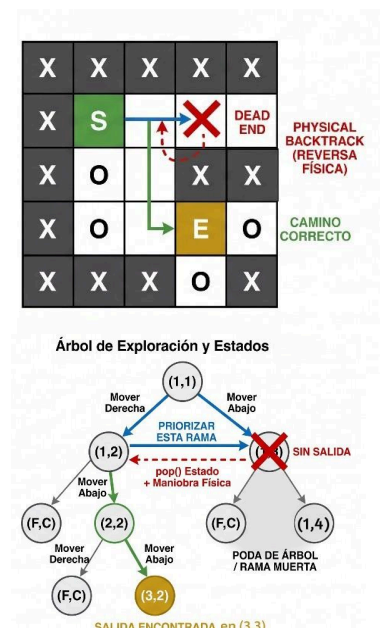
3. Estando en (1, 2), el ciclo for de 4 direcciones evalúa el entorno. Supongamos que la aspiradora sigue mirando a la Derecha y ve la celda (1, 3). Como está vacía y no visitada, decide avanzar hacia allí, priorizando esa rama.

Pila: [(1,1,3,0), (1,2,3,1), (1,3,3,2)] | Visitados: {(1,1), (1,2), (1,3)}

4. En la celda (1, 3), al finalizar el ciclo de evaluación, la variable moved_this_turn permanece en False. Se ha detectado una rama muerta. Al no poder avanzar, el algoritmo debe "podar" este camino y regresar al estado anterior (BACKTRACKING).

Pila: [(1,1,3,0), (1,2,3,1)]

5. De nuevo en (1, 2), el algoritmo continúa el ciclo de búsqueda que había quedado pausado. Eventualmente gira para mirar hacia Abajo (Dirección = 2), descubre la celda libre en (2, 2) y avanza por la rama correcta hasta encontrar la salida 'S'.



Pseudocódigo

```
encontrar_salida (profundidad_actual, limite_profundidad) {
    // chequeo de depth limit al principio
    si (profundidad_actual es mayor o igual a limite_profundidad) entonces {
        retornar FALLA
    }
    marcar la celda actual como "visitada"
    // los giros para avanzar por una rama en la búsqueda
    para cada giro DESDE 1 HASTA 4 HACER {
        celda_frente = mirar la celda directamente enfrente
        // El chequeo de la salida ahora está adentro para revisar las 4
        direcciones
        si (celda_frente es igual a 'E') entonces {
            avanzar
            retornar EXITO
        }
        si (celda_frente es igual a 'O') Y (celda_frente no está en
"visitadas") entonces {
            direccion_entrada = obtener dirección actual de la aspiradora
            avanzar
            // LLAMADA RECURSIVA: explorar recursivamente desde esta nueva
            celda
            si encontrar_salida_dls(profundidad_actual + 1,
limite_profundidad) es igual a EXITO entonces {
                retornar EXITO // ¡listo!
            }
            // BACKTRACKING: deshacer
            // si la exploración de arriba falló, es un callejón sin salida.
            retrocede físicamente
            // realinea la aspiradora a la dirección con la que entró a esta
            celda
            mientras (dirección actual no sea igual a direccion_entrada)
HACER {
                girar izquierda
            }
            // retrocede físicamente
            girar izquierda
            girar izquierda
            avanzar
            girar izquierda
            girar izquierda
        }
        // rota para explorar otra rama (esto sería parte del podado,
"olvidando" el camino sin salida del que retrocedió
        girar izquierda
    }
    // sin solución luego de las búsquedas: se giró en un círculo completo,
se revisó todo y no se encontró nada
    retornar FALLA
}
```

ANÁLISIS DE COMPLEJIDAD

El análisis se basa en el modelado con un grafo $G = (C, U)$. Definimos C como la cantidad de casillas libres transitables en el laberinto de dimensiones $N \times M$, y U como la cantidad de uniones o pasos válidos entre dichas casillas. En una grid bidimensional bidireccional, cada casilla tiene a lo sumo 4 vecinos, por lo que sabemos que $U \leq 4C$.

Ambas implementaciones del algoritmo comparten la misma complejidad temporal teórica de

$$O(C + U)$$

Sólo difieren en su manejo espacial y en la robustez frente a laberintos de gran escala.

Complejidad Temporal

El algoritmo, al ser un DFS con un registro de nodos visitados, explora cada vértice $c \in C$ y cada arista $u \in U$ a lo sumo una vez en la etapa de avance, y una vez en la etapa de retroceso.

En un DFS tradicional, visitar un vértice cuesta $O(1)$. Para esta implementación, "visitar" implica operaciones físicas de la aspiradora (rotar a la izquierda, avanzar y deshacer el camino). Sin embargo, el costo máximo de procesar una casilla está acotado por valores constantes:

- Revisar las 4 direcciones toma a lo sumo 4 giros.
- Deshacer un paso (backtracking) requiere a lo sumo 2 giros para darse vuelta, 1 avance y 2 giros para recuperar la orientación original.

Como el número máximo de operaciones por casilla es un valor constante independiente del tamaño del laberinto, las constantes se desestiman. Por lo tanto, la complejidad temporal en el peor de los casos (donde la aspiradora debe recorrer todo el laberinto antes de "darse cuenta" de que no hay salida, o de encontrarla en la última casilla) es

$$O(C + U)$$

Dado que $U \leq 4C$, la complejidad se simplifica a

$$O(C)$$

Y como en el peor de los casos todas las casillas de la matriz $N \times M$ están vacías, la cota superior del tiempo de ejecución está dada por

$$O(N \times M)$$

Complejidad Espacial

El costo de memoria del algoritmo está dictado por las dos estructuras de datos que deben mantenerse vivas durante la ejecución: la memoria de casillas visitadas y la pila de exploración.

- Hash de Visitados: En el peor caso, si la aspiradora visita todas las casillas libres del laberinto, el conjunto *visited* deberá almacenar C tuplas de coordenadas. Por lo tanto, su complejidad espacial es $O(C)$.
- Call Stack: El tamaño de la pila depende de la profundidad máxima de la rama que se esté explorando. Esto significa que la pila puede crecer hasta tener un tamaño igual a la cantidad de casillas transitables, es decir, $O(C)$.

Sumando ambos requerimientos

$$(O(C) + O(C))$$

la cota asintótica total para la memoria requerida es

$$O(N \times M)$$

Pero, por DLS...

Con la implementación de la Búsqueda Limitada por Profundidad (DLS), la complejidad espacial en el peor de los casos ya no está estrictamente atada a la cantidad total de celdas libres C . El tamaño máximo de la pila de exploración (tanto el Call Stack en recursiva como la lista en iterativa) queda estrictamente acotado por el límite de profundidad L . Por lo tanto, la complejidad espacial máxima es

$$O(\min(C, L))$$

O, lo que es igual

$$O(\min(N \times M, L))$$

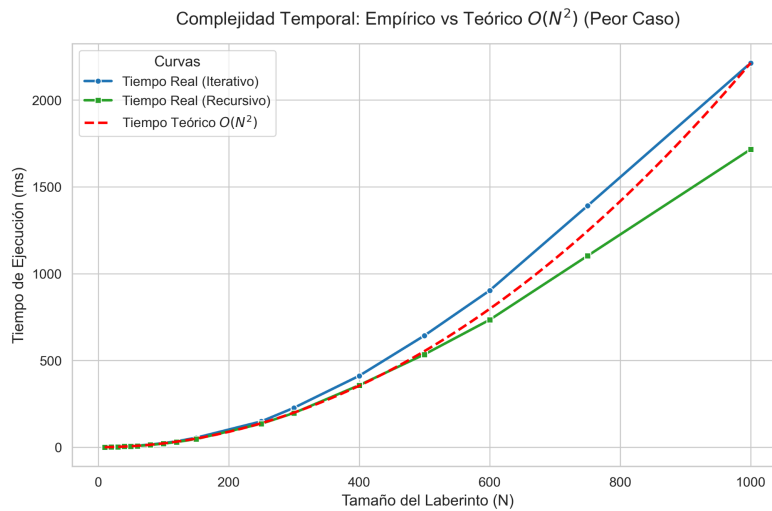
DLS también optimiza la cota temporal en la práctica. Al restringir la exploración a una profundidad máxima L , el algoritmo no necesariamente recorrerá los C vértices del laberinto, sino únicamente el subconjunto de vértices alcanzables dentro de ese límite (C_L). Por lo tanto, la complejidad temporal estricta de la implementación pasa de ser

$$O(C_L)$$

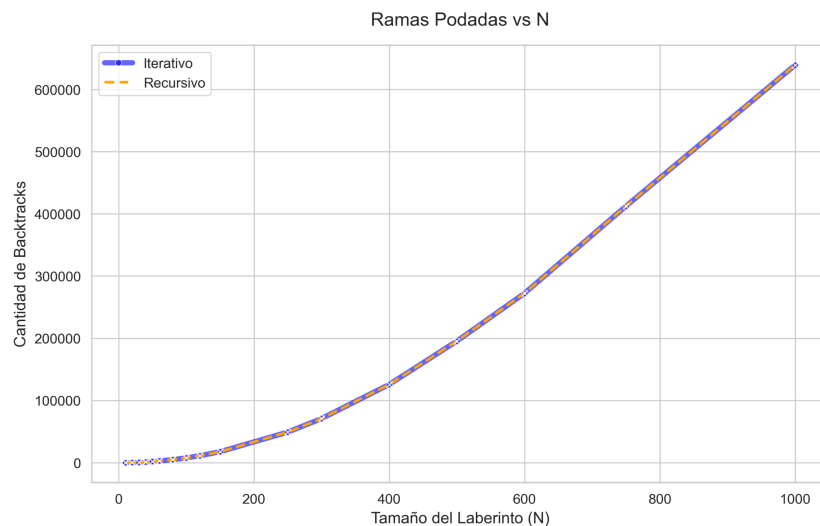
RESULTADOS Y CONCLUSIONES

Para el análisis de datos obtenidos, se realizó un benchmarking corriendo el algoritmo con diferentes tamaños de laberintos, cuadrados (variando N con dimensiones $N \times N$) para validar las hipótesis teóricas planteadas. Además, para mantener la consistencia entre laberintos, la aspiradora siempre empezó su recorrido desde el centro de la grilla.

Según el gráfico de Tiempo de Ejecución vs. Cota Teórica, el comportamiento empírico de los algoritmos se ajusta perfectamente a la curva esperada de $O(N^2)$ ($C = N^2$). Al ejecutar escenarios sin solución (el peor caso algorítmico, donde se explora y poda la totalidad del espacio de búsqueda), la cantidad de operaciones físicas de la aspiradora crece de manera proporcional.

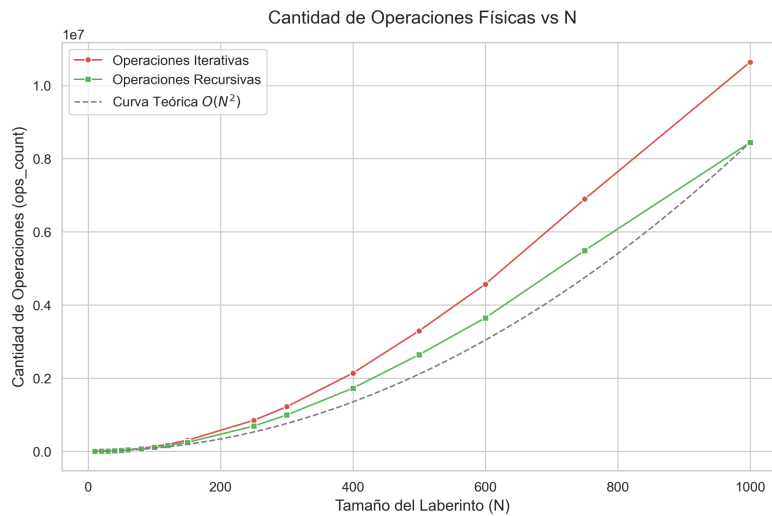


Contrario a la intuición inicial de que una versión iterativa podría ser más rápida, los resultados demuestran un rendimiento temporal casi idéntico. Esto se debe a que, lógicamente, ambos algoritmos construyen el mismo árbol de exploración y efectúan la misma cantidad exacta de backtracks (ramas podadas). El costo de gestionar la pila manual mediante métodos como `append()` y `pop()` en Python equipara el costo de la sobrecarga del Call Stack en la versión recursiva.

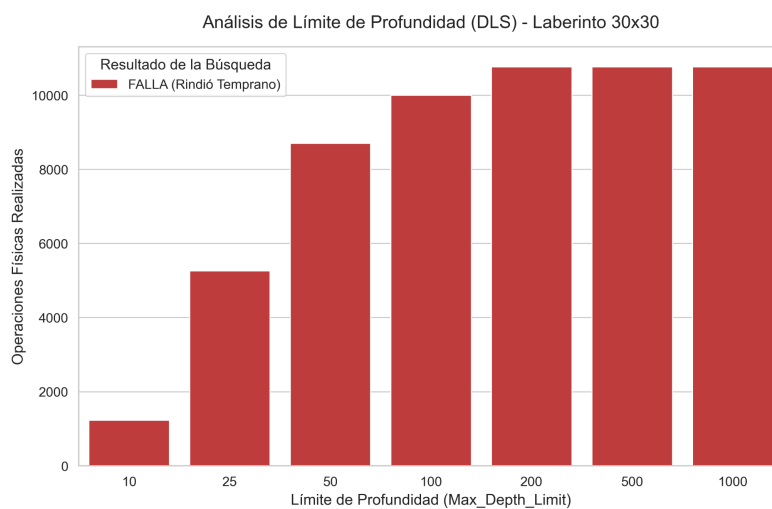


Durante el benchmarking, fue estrictamente necesario alterar los límites del entorno de ejecución (mediante `sys.setrecursionlimit()`) para evitar que el algoritmo recursivo tire un error de Stack Overflow al procesar laberintos de $N \geq 60$. La solución iterativa, en cambio, gestionó dinámicamente su memoria en el Heap, demostrando ser la única opción viable y segura para un entorno de producción real sin requerir alteraciones en la configuración del lenguaje.

Al analizar un laberinto sin solución de tamaño $N = 30$, se observa que la cantidad de operaciones crece linealmente con el límite de profundidad hasta alcanzar un punto de



saturación. A partir de este punto, la curva se aplana por completo. Esto demuestra que, debido a la densidad de obstáculos (manejado con una variable `wall_probability`), el componente conexo máximo del grafo es significativamente menor al peor caso teórico de $N^2 = 900$ en el que todo el grafo llega a explorarse. Aumentar el límite de profundidad más allá de este umbral no genera impacto, ya que el algoritmo logra agotar el espacio de búsqueda por completo.



En conclusión, el desarrollo de este problema demostró que modelar el laberinto como un grafo implícito, con la elección de una Búsqueda en Profundidad (DFS) fue la mejor abstracción para un agente físico como la aspiradora.

El análisis empírico demostró que la introducción de técnicas como la Búsqueda Limitada por Profundidad (DLS) es fundamental para la optimización en entornos reales. Por otro lado, demuestra que la solución propuesta respeta la cota teórica asintótica y al mismo tiempo resuelve las limitaciones de un sistema que interactúa con el mundo físico.

Problema 4 – Programación Dinámica

SUPUESTOS, CONDICIONES Y LIMITACIONES

En primer lugar, se define el problema a resolver. Se busca que dada una cadena de largo n , se calcule la menor cantidad posible de palíndromos contiguos que se pueden formar incluyendo todos sus caracteres.

Se asume por anticipado que no es necesario normalizar la cadena ingresada de ninguna manera, por ejemplo no se normalizan mayúsculas y minúsculas y si hubiera puntuación se toma como un carácter más.

Una modificación a los algoritmos dinámicos típicos es que se pre-genera y utiliza una tabla de los palíndromos que se pueden hallar dentro de cada palabra empezando por cada una de sus letras para resolver el problema, por lo tanto una limitación surgiría en términos del uso de memoria al lidiar con cadenas más largas.

DISEÑO

Definiendo $M[i]$ como el mínimo número de palíndromos necesarios para descomponer la cadena conformada por los primeros i caracteres, **la ecuación de recurrencia sería:**

$$M[i] = \min(M[j] + 1), \text{ para todo } 0 \leq j < i, \text{ palabra}[j:i] \text{ es palíndromo}$$

Se observan tanto la presencia de la característica de subestructura óptima como la de problemas superpuestos¹.

Subestructura óptima: La solución para una palabra completa contiene a todas las soluciones ($M[i]$) para todas las cadenas más cortas formadas por las letras hasta la posición i en la palabra.

Problemas superpuestos: Para cada cadena, la solución óptima puede contener a la solución óptima previamente calculada para sus componentes

Se utiliza **memoization** al ir llenando M con el mínimo número de palíndromos hasta cada letra de la palabra.

Pseudocódigo

```
funcion MinimoPalindromos(cadena):  
    n = longitud(cadena)  
    si n = 0:  
        retornar 0  
  
    PAL = matriz n x n con false  
    para i = 0 hasta n-1:  
        PAL[i][i] = true
```

```

    para i = 0 hasta n-2:
        PAL[i][i+1] = (cadena[i] == cadena[i+1])

    para largo == 3 hasta n:
        para i = 0 hasta n-largo:
            j = i + largo - 1
            PAL[i][j] = (cadena[i] == cadena[j]) y
PAL[i+1][j-1]

    M = arreglo tamaño n+1 con infinito
    M[0] = 0

    para i = 1 hasta n:
        para j = 0 hasta i-1:
            si PAL[j][i-1] = true:
                M[i] = min(M[i], M[j] + 1)

    devolver M[n]

```

Las estructuras de datos utilizadas fueron las siguientes:

- Matriz booleana PAL n x n que almacena si una subcadena es un palíndromo
- Array M (tamaño n+1) que almacena la mínima cantidad de palíndromos para cada prefijo o subcadena previa a la ubicación actual en el array.

SEGUIMIENTO

Tomamos el ejemplo de “ARACALACANA” para realizar el seguimiento

Seguimiento de PAL

Durante el preprocesamiento, se marcan como verdaderas las subcadenas palindrómicas.

Entre las más importantes para el resultado final aparecen:

- ARA (posiciones 0..2)
- CALAC (posiciones 3..7)
- ANA (posiciones 8..10)

También aparecen todos los palíndromos de longitud 1 en cada posición y otros palíndromos intermedios.

Seguimiento de M

Se utiliza M[i] como mínimo número de palíndromos para el prefijo de longitud i.

- M[0] = 0
- M[1] = 1 (A)
- M[2] = 2 (A | R)

- $M[3] = 1$ (ARA)
- $M[8] = 2$ (ARA | CALAC)
- $M[11] = 3$ (ARA | CALAC | ANA)

Con esto, el algoritmo devuelve $M[11] = 3$.

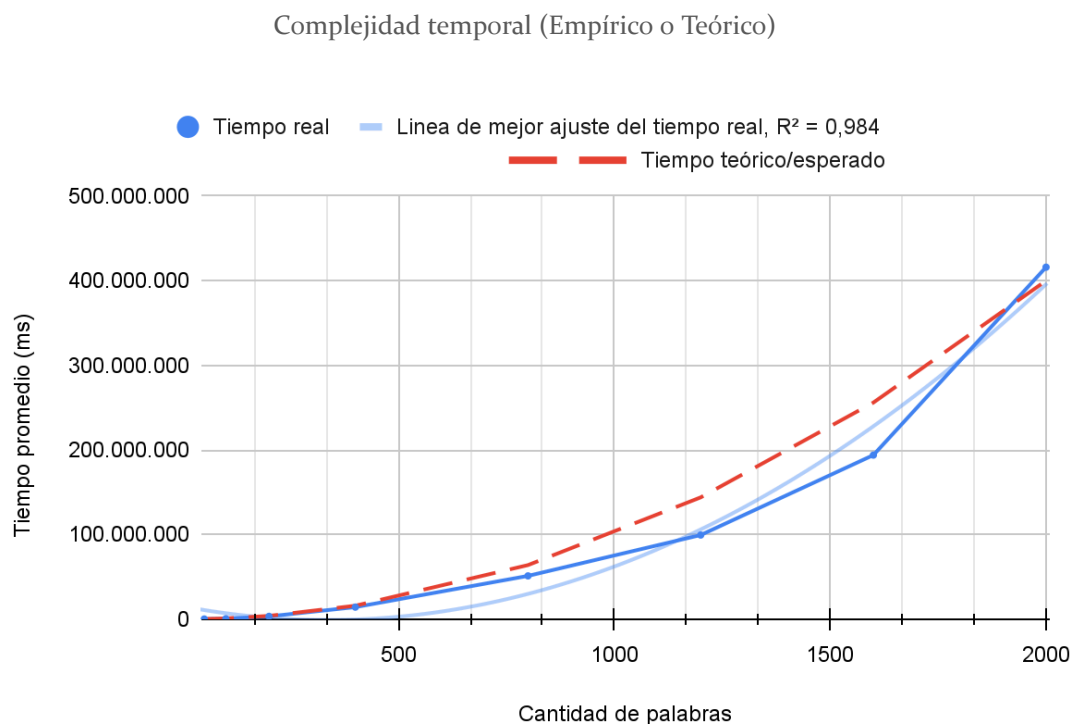
ANÁLISIS DE COMPLEJIDAD

El algoritmo principal tiene una complejidad temporal de $O(n^2)$ ya que para cada i hay que recorrer todos los j anteriores. Además la construcción de la matriz PAL también es de complejidad $O(n^2)$

SETS DE DATOS

Los sets de datos utilizados son conjuntos de cadenas generadas aleatoriamente con una semilla aleatoria fija (12345) de tamaños $n = 50, 100, 200, 400, 800, 1200, 1600, 2000$

TIEMPOS DE EJECUCIÓN



Análisis de los tiempos de ejecución

Como se puede observar, los tiempos reales de ejecución coincidieron con lo esperado teóricamente, ya que se va observando una curva cuadrática entre los resultados observados. Los números son algo menores y la curva no es perfecta debido a variaciones del entorno de ejecución.

Referencias bibliográficas

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.
2. Kleinberg, J., & Tardos, É. (2006). Algorithm Design. Pearson.
3. Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1, 269–271.